



REPORTE DE PRÁCTICA: SEGG- U2-P4H-2

Valeria Hernández Cortés

Grupo IDGS17

Análisis del archivo de configuración de Promtail

Sección	Finalidad dentro de la arquitectura
server	Define los puertos de red donde el propio Promtail expone sus métricas internas y su estado.
clients	Especifica la URL del servidor destino al cual Promtail enviará los flujos de registros procesados.
positions	Define la ruta de un archivo temporal (positions.yaml) donde el agente guarda un registro exacto de la última línea que leyó de cada archivo para evitar duplicidad.
scrape_configs	Es el bloque principal de ejecución. Contiene la lista de <i>jobs</i> que dictan de dónde y cómo se extraerán los datos.
static_configs	Define objetivos de recolección fijos y permite asignar un conjunto de etiquetas base a todos los logs provenientes de ese origen.
labels	Son los metadatos que se adjuntan a los registros. Sirven como índices para estructurar la base de datos en Loki.

Configuración del monitoreo de múltiples archivos

Para configurar Promtail de manera que sea capaz de leer múltiples archivos generados por VulnerableApp se utilizó un wildcard en la ruta de búsqueda: `__path__: /var/log/applogs/*.txt`. A diferencia de Seq, Promtail no recibe los registros mediante la red. Su función es actuar como un agente que monitorea directamente los archivos de log almacenados en el sistema operativo, leyendo automáticamente las nuevas líneas que se agregan conforme la aplicación las genera. Para que esto sea posible cuando Promtail se ejecuta dentro de un contenedor, es necesario indicar la ubicación exacta de los archivos de log del equipo anfitrión. Esto se logró mediante un bind mount, el cual comparte un directorio del sistema anfitrión con el contenedor y permite que Promtail acceda a los archivos en tiempo real.

Enriquecimiento de registros mediante labels

Las labels (etiquetas) son datos que Promtail agrega a cada registro antes de enviarlo a Loki. Su función es clasificar los logs para que después sea más fácil buscarlos y filtrarlos. En esta configuración, Promtail analiza el contenido de cada registro mediante reglas y expresiones regulares para asignar automáticamente las siguientes etiquetas:

- Application: Identifica errores internos del sistema o excepciones no controladas
- Security: Identifica eventos relacionados con la autenticación y control de acceso
- Audit: Identifica registros informativos o de trazabilidad sobre el uso general del sistema

El uso de etiquetas hace que las consultas sean mucho más rápidas. En lugar de revisar línea por línea todos los archivos de log, Loki utiliza las etiquetas para encontrar únicamente los registros que cumplen con la búsqueda solicitada. Esto reduce el tiempo de respuesta y el consumo de recursos del sistema.

Seq vs Grafana + Loki

Característica	Seq	Grafana + Loki
Búsqueda de registros	Permite buscar fácilmente los logs utilizando las propiedades generadas por Serilog.	Permite realizar búsquedas avanzadas mediante LogQL, filtros y expresiones regulares.
Filtros por etiquetas (labels)	No utiliza labels; organiza los registros mediante las propiedades del log.	Las labels son la base para organizar y filtrar los registros de forma rápida.
Dashboards	Incluye paneles básicos para visualizar eventos y errores.	Permite crear paneles completos y personalizables, combinando logs con otras métricas.
Consultas	Utiliza una sintaxis sencilla para buscar y filtrar información.	Utiliza LogQL, que permite realizar consultas, filtros y análisis más avanzados.
Escalabilidad	Adecuado para proyectos pequeños o para el entorno de desarrollo.	Diseñado para manejar grandes volúmenes de registros en entornos de producción y sistemas distribuidos.

Uso principal	Facilita el análisis y la depuración de aplicaciones durante el desarrollo.	Se utiliza para el monitoreo y la observabilidad de aplicaciones e infraestructura en producción.
---------------	---	---

Conclusiones de la comparativa: Seq es una herramienta muy útil para el desarrollo de aplicaciones, ya que se integra fácilmente con Serilog y permite encontrar errores y revisar los registros de forma rápida. En cambio, Grafana junto con Loki requiere un poco más de configuración y aprendizaje, especialmente para usar LogQL y las labels, pero ofrece muchas más posibilidades para monitorear aplicaciones en producción. Además de analizar logs, permite centralizar la información de diferentes sistemas en un solo lugar y realizar consultas más eficientes, lo que la convierte en una solución más completa para la observabilidad de aplicaciones e infraestructura.

Pregunta de reflexión: ¿Qué ocurriría si Promtail no almacenara la posición del último registro leído en positions.yaml?

Si Promtail no almacenara la información en el archivo positions.yaml, perdería el registro de la última línea de log que había leído. Como resultado, cada vez que el servicio se reiniciara no sabría desde dónde continuar la lectura de los archivos. Podría volver a leer el archivo de log desde el inicio y enviar nuevamente los mismos eventos a Loki, generando información repetida y si comenzara a leer únicamente los eventos nuevos después del reinicio, los registros generados mientras Promtail estuvo detenido no serían enviados a Loki.

Evidencias solicitadas:

```
promtail-config.yaml
1  server:
4
5  positions:
6    filename: /tmp/positions4.yaml
7
8  clients:
9    - url: http://loki:3100/loki/api/v1/push
10
11  scrape_configs:
12    - job_name: vulnerableapp_logs    "vulnerableapp": Unknown word.
13      static_configs:
14        - targets:
15            - localhost
16          labels:
17            job: vulnerableapp    "vulnerableapp": Unknown word.
18            env: development
19            __path__: /var/log/applogs/*.txt    "applogs": Unknown word.
20
21      pipeline_stages:
22        - regex:
23            expression: '(?P<Security>EVENTO AUTH|inválidas)'    "EVENTO": Unknown word.
24          labels:
25            Security:
26        - regex:
27            expression: '(?P<Application>Error|Exception)'
28          labels:
29            Application:
30        - regex:
31            expression: '(?P<Audit>\\[INF\\]|Inicio)'    "Inicio": Unknown word.
32          labels:
33            Audit:
```

Archivo promtail-config.yml actualizado

```
C:\Users\valec>cd C:\Users\valec\OneDrive\Documentos\git\VulnerableApp
C:\Users\valec\OneDrive\Documentos\git\VulnerableApp>docker ps
```

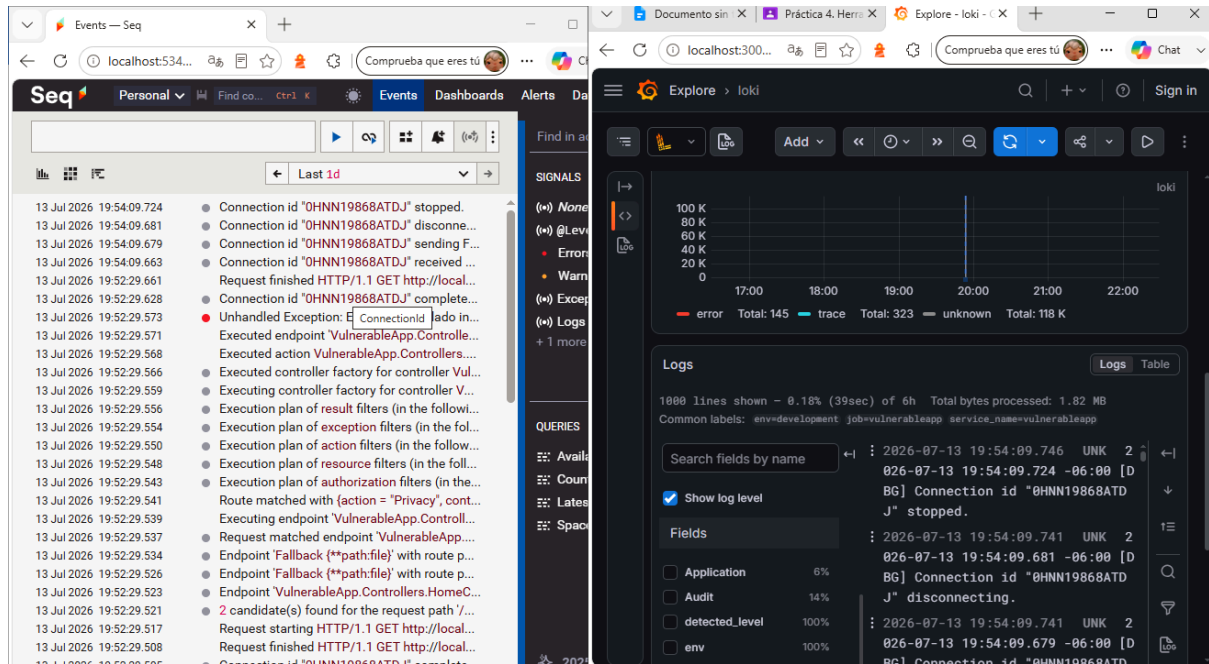
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
8b3d1e3b0420	grafana/grafana:latest	"/run.sh"	46 hours ago	Up 7 minutes	0.0.0.0:3000->3000/tcp	grafana
5441b91f9d45	grafana/promtail:latest	"/usr/bin/promtail --"	46 hours ago	Up 7 minutes	0.0.0.0:3100->3100/tcp	promtail
827fc4953c51	grafana/loki:latest	"/usr/bin/loki -conf_"	46 hours ago	Up 7 minutes	0.0.0.0:3100->3100/tcp	loki
a142a8e5e516	datahust/seq	"/bin/seqentry"	11 days ago	Up About a minute	443/tcp, 5341/tcp, 45341/tcp, 0.0.0.0:5341->80/tcp	seq

Todos los servicios en ejecución

```
C:\Users\valec\OneDrive\Documentos\git\VulnerableApp>docker compose restart promtail
[+] Restarting 1/1
[+] Container promtail Started
```

Promptail restablecido para guardar los cambios realizados

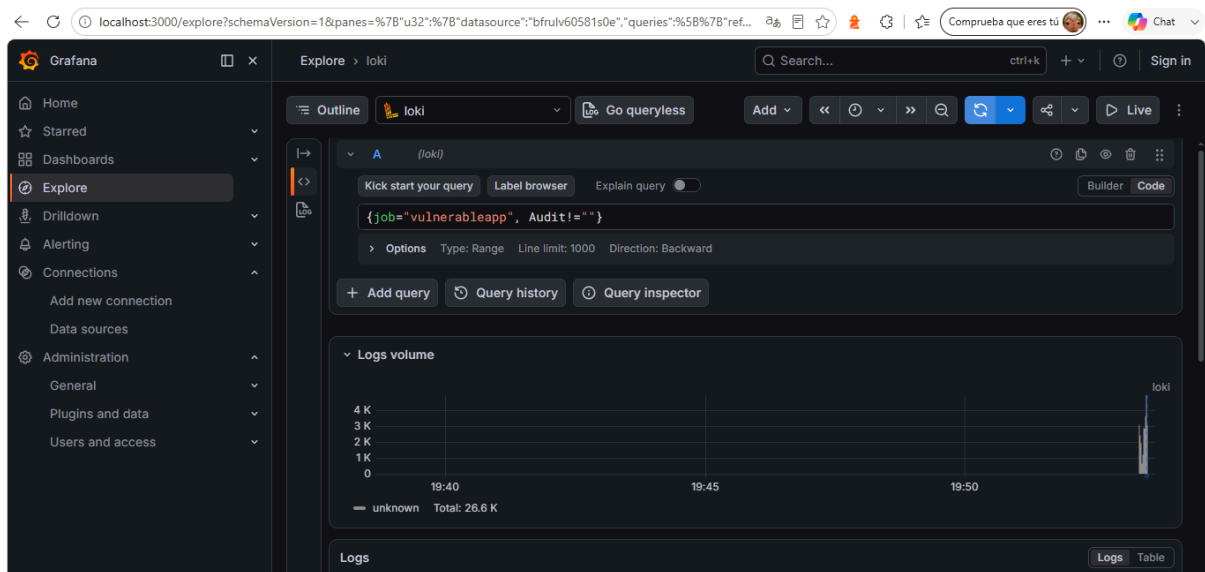
Reporte de Práctica: Herramientas para Administración de Logs. Parte 2



Seq coexistiendo con Grafana

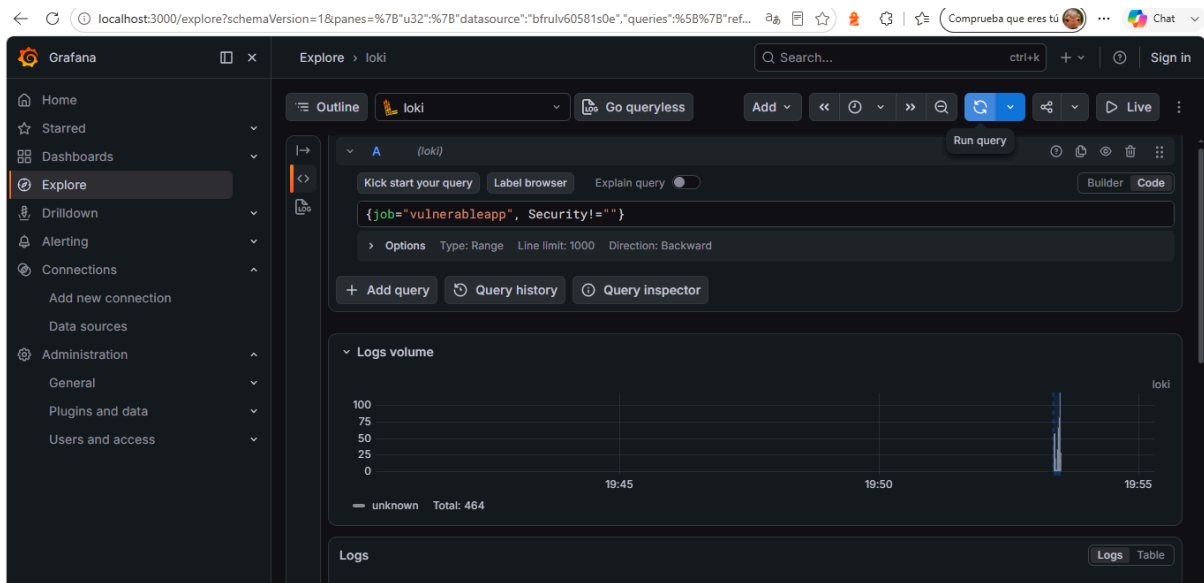
Grafana mostrando consultas exitosas por labels

Funcionamiento correcto de loki mediante el filtrado de Audit:



Filtrado Security:

Reporte de Práctica: Herramientas para Administración de Logs. Parte 2



Filtrado application:

